

[Open in app](#) ↗[Sign up](#)[Sign in](#)**Medium**

Search



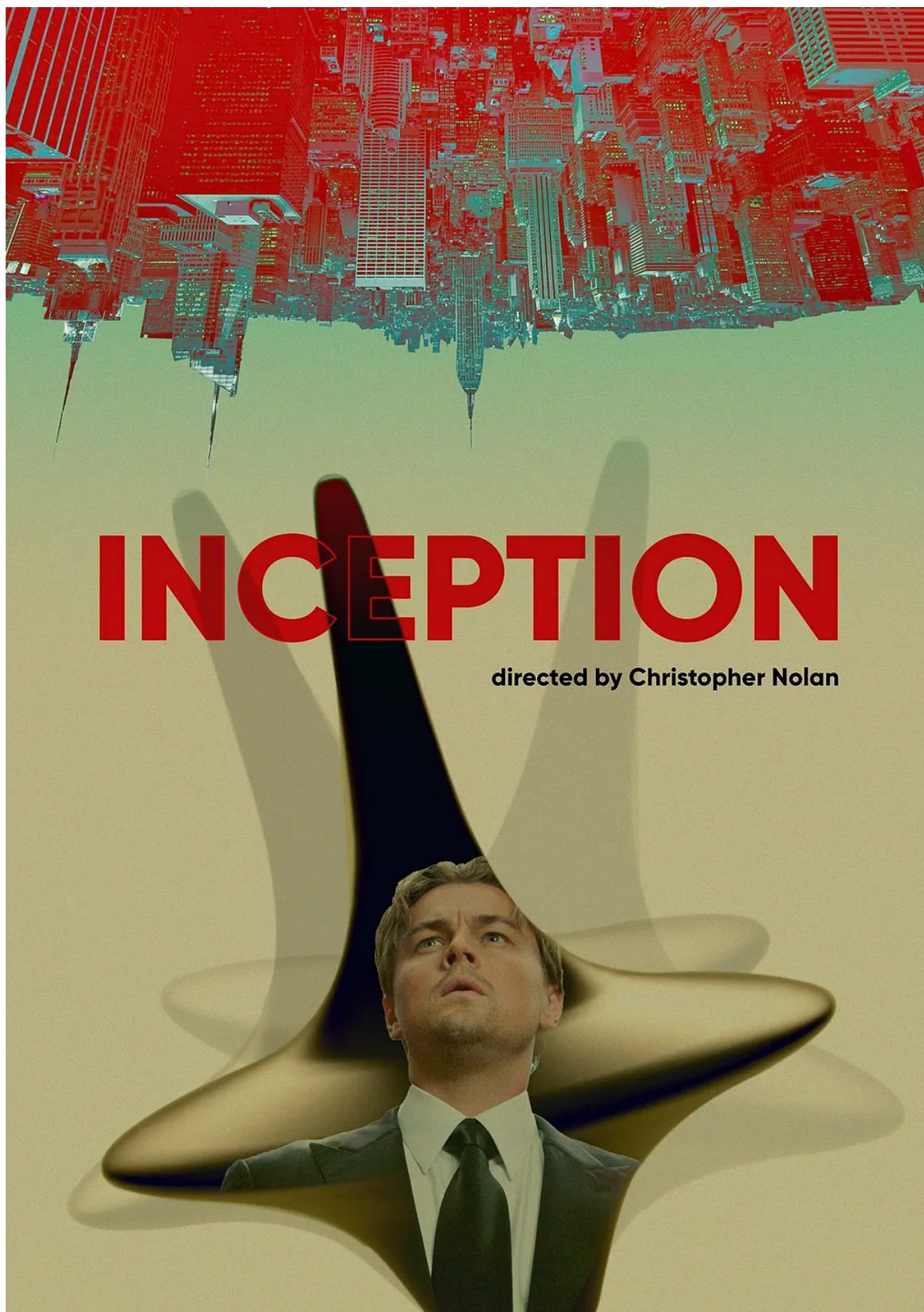
Inception 42: A Comprehensive Guide to Dockerizing Your First Infrastructure(PART I)

7 min read · May 12, 2026



Abdilah CH

[Follow](#)[Listen](#)[Share](#)



Most students fail the Inception project not because Docker is hard, but because

they skip the system design layer and jump directly into writing Dockerfiles. The real difficulty is not “running containers”, it is understanding how isolated processes cooperate as a distributed system, and how data, networking, and process lifecycles are structured across containers.

In this project, you are not building a single application. You are building a small infrastructure composed of independent services that must behave like a coherent production stack. Each service has a strict responsibility boundary, and Docker is only the mechanism that enforces isolation and communication rules between them.

At the core, your architecture is a classic three-tier web system:

```
graph TD
    UserBrowser[User Browser] -- "HTTPS (port 443)" --> NGINX[NGINX (reverse proxy)]
    NGINX -- "FastCGI (port 9000)" --> WordPress[WordPress (PHP-FPM)]
    WordPress -- "SQL (port 3306)" --> MariaDB[MariaDB]
```

This diagram is not just a visual aid. It defines the entire flow of execution, request handling, and data persistence. Every container in this chain must be designed around this flow, not independently.

NGINX is your only public entry point. It terminates HTTPS connections and acts as a reverse proxy. That means it receives encrypted traffic from the browser, decrypts it (TLS termination happens here), and forwards the request internally to the WordPress service. No other container is exposed to the outside world. This is a critical security and architecture constraint.

Internet

```

|
| TLS handshake (HTTPS termination)
v
NGINX container
|
| internal Docker network
v
WordPress (PHP-FPM)
```

WordPress is not a full web server in this setup. It runs PHP-FPM, which is a FastCGI process manager. It does not handle HTTP directly. It only executes PHP scripts when NGINX forwards requests. This separation is intentional: NGINX handles networking, WordPress handles application logic.

```
NGINX -> FastCGI request -> PHP-FPM process (WordPress)
```

Below WordPress is MariaDB, which is purely a database service. It is not accessible from the outside world. Only WordPress can communicate with it over the internal Docker network using port 3306. This separation ensures that the database is isolated from direct external attack vectors.

```
WordPress
|
| SQL queries (3306)
v
MariaDB
```

Now the most important conceptual layer: persistence vs ephemerality.

Containers are ephemeral by default. If a container is destroyed, everything inside it disappears. This includes your WordPress installation files and your database state. That is why volumes exist. They are external storage systems attached to containers to preserve data beyond container lifecycle.

- one for WordPress files (themes, uploads, configuration)

- one for MariaDB database data

```
[WordPress container] ---> [wordpress_volume]
[Mariadb container] ---> [mariadb_volume]
```

Without these volumes, your system would reset every time you restart Docker, which is unacceptable for any real infrastructure.

Another critical concept is the Docker network. All containers communicate through a private virtual network created by Docker. They do not use localhost, nor physical IPs. Instead, they resolve each other by container name.

```
docker network (internal bridge)

NGINX <----> wordpress <----> mariadb
```

This means NGINX can reach WordPress simply using `wordpress:9000`, and WordPress can reach MariaDB using `mariadb:3306`. Docker's embedded DNS system handles this resolution automatically.

Finally, startup order matters because services depend on each other. MariaDB must be ready before WordPress tries to connect. WordPress must be ready before NGINX starts forwarding traffic. However, Docker itself does not guarantee strict ordering, which means you must design for retry logic or health checks instead of relying on startup sequence.

Expected boot logic:

1. mariadb starts → database initialized
2. wordpress starts → waits for DB connection
3. nginx starts → waits for wordpress upstream
4. traffic begins

A subtle but important detail is PID 1 inside containers. Each container runs a main process as PID 1. If this process crashes or misbehaves, the container stops. This is why services like PHP-FPM, NGINX, and MariaDB must be run in foreground mode (`daemon off` , `-F` , etc.). If you run them in background mode, the container will exit immediately.

In summary, Inception is not about Docker commands. It is about designing a minimal production-grade architecture with strict boundaries:

- NGINX handles external communication and TLS
- WordPress handles application logic
- MariaDB handles persistent data
- Docker network handles internal communication
- Volumes handle persistence
- PID 1 defines container lifecycle

If you understand this diagram and these responsibilities, the project becomes a structured system design exercise rather than a scripting task.

Global Architecture (Execution Flow)

Your system is a strictly layered request pipeline:

```
Client (Browser)
  ↓ HTTPS (TLS over port 443)
NGINX (edge reverse proxy)
  ↓ FastCGI (TCP 9000)
WordPress (PHP-FPM runtime)
  ↓ SQL protocol (TCP 3306)
MariaDB (persistent database engine)
```

Key interpretation

This is not “four containers talking”.

It is a **request transformation chain**:

- HTTP request → TLS termination → reverse proxy
- static routing vs dynamic PHP execution
- application logic execution
- database query execution

Each hop changes the protocol layer.

PART 2 — Container Responsibilities (Strict Separation of Concerns)

A. NGINX container (Edge Layer)

NGINX is your **only** external-facing service.

It is responsible for:

- TLS termination (HTTPS decryption)
- HTTP request parsing
- static file serving (if any)
- forwarding PHP requests to FastCGI backend

Important constraint:

NGINX does NOT interpret PHP.

It only forwards:

```
.php request → php-fpm backend
```

Mechanism:

```
NGINX → wordpress:9000 (FastCGI protocol)
```

So NGINX is a **traffic router + TLS gateway**, not an application runtime.

B. WordPress container (Application Layer)

WordPress container is misnamed architecturally. It is actually:

PHP-FPM runtime + WordPress codebase

It contains:

- php-fpm (process manager)
- PHP interpreter
- WordPress source files

It does NOT include:

- nginx
- database
- TLS logic

It listens internally on:

```
port 9000 (FastCGI socket)
```

Role:

- execute PHP scripts
- generate dynamic responses
- communicate with database

So it is an **application execution engine**, not a web server.

C. MariaDB container (Data Layer)

MariaDB is completely isolated.

It contains:

- database engine only
- storage engine (InnoDB, etc.)
- SQL parser/executor

Responsibilities:

- persist WordPress state
- handle queries (users, posts, metadata)

No awareness of:

- HTTP
- PHP
- Docker networking logic

It only understands:

SQL over TCP 3306

PART 3 — Docker Network (Service Discovery Layer)

All containers communicate via a **private virtual bridge network**.

Core idea:

Docker provides built-in DNS resolution.

So instead of IPs:

```
wordpress:9000  
mariadb:3306
```

becomes possible automatically.

Internal resolution flow:

```
NGINX container  
  ↓ resolves "wordpress"  
Docker DNS  
  ↓ returns container IP  
WordPress container (9000)
```

All services **MUST** share the same network.

Get Abdilah CH's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐

Remember me for faster sign in

Otherwise:

- no name resolution
- no inter-container communication
- architecture breaks completely

So Docker network is your **internal infrastructure backbone**, not optional glue.

PART 4 — Persistent Storage (State vs Ephemeral Reality)

Containers are stateless by design.

Meaning:

- container dies → filesystem dies
- restart → fresh environment

So persistence must be externalized.

You use **named volumes**:

Two required persistent layers:

```
/home/login/data/mariadb → database files  
/home/login/data/wordpress → uploaded content + WP state
```

Logical mapping:

```
MariaDB container → mariadb volume  
WordPress container → wordpress volume
```

Without volumes:

- database resets on restart
- WordPress loses uploads
- system becomes unusable

So volumes are not optional storage — they are **state survival mechanisms**.

PART 5 — Why Bind Mounts Are Forbidden

Bind mounts expose host filesystem directly:

```
/host/path → container
```

Problem in this subject:

- breaks portability
- bypasses Docker lifecycle management
- introduces host dependency
- reduces reproducibility

Named volumes instead:

- are managed by Docker engine
- abstract physical location
- integrate with backup/restore lifecycle
- enforce isolation boundary

So restriction is intentional: it forces **infrastructure discipline**, not convenience.

PART 6 — HTTPS and Security Boundary

Only one public endpoint exists:

```
NGINX :443
```

Everything else is internal:

```
✗ 3306 (MariaDB exposed) → forbidden  
✗ 9000 (PHP-FPM exposed) → forbidden
```

Correct exposure model:

```
Public:
  443 → NGINX only

Private:
  9000 → WordPress only (internal network)
  3306 → MariaDB only (internal network)
```

So NGINX acts as a **security perimeter gateway**.

PART 7 — Why “infinite loops” are forbidden

Patterns like:

```
tail -f
sleep infinity
while true
```

are anti-patterns because they fake service lifecycles.

A container must run a **real foreground process**.

Correct model:

```
nginx -g "daemon off;"
php-fpm -F
mariadb --foreground
```

Reason:

- PID 1 must be the actual service process
- container lifecycle = process lifecycle

- no artificial keep-alive hacks

PART 8 — PID 1 (Critical Concept)

Inside a container:

```
PID 1 = main process
```

PID 1 is responsible for:

- signal handling (SIGTERM, SIGKILL)
- child process reaping
- graceful shutdown

Bad PID 1 behavior causes:

- zombie processes
- stuck containers
- improper restart behavior

So every service must run in **foreground mode intentionally**, not background daemon mode.

PART 9 — Environment Preparation (Host Layer)

Before Docker architecture:

Install Docker stack

- Docker Engine
- Docker Compose plugin

Verify:


```
docker --version  
docker compose version
```

Enable daemon:

```
systemctl enable docker  
systemctl start docker
```

Add user to docker group:

```
usermod -aG docker $USER
```

This removes need for sudo.

PART 10 — Project Structure (Source Layer)

Canonical structure:

```
~/Inception/srcs/  
  requirements/  
    nginx/  
    wordpress/  
    mariadb/
```

Each service contains:

- Dockerfile
- configuration files
- optional scripts

This enforces **service isolation at filesystem level**, not only runtime level.

PART 11 — Persistent Storage Directories

Host-level storage:

```
/home/login/data/mariadb  
/home/login/data/wordpress
```

These map to Docker volumes.

So host becomes **physical storage backend**, Docker manages abstraction.

PART 12 — Domain Resolution (Local DNS Simulation)

Edit:

```
/etc/hosts
```

Add:

```
127.0.0.1 yourlogin.42.fr
```

This simulates:

- real DNS resolution
- production domain routing
- TLS hostname binding

So browser resolves:

```
yourlogin.42.fr → localhost → nginx
```

PART 13 — Validation Layer (Docker sanity)

Test runtime:

```
docker run hello-world
```

This confirms:

- daemon works
- image pull works
- container execution works



Follow

Written by Abdilah CH

17 followers · 1 following

Student at 1337, passionate about coding, problem-solving, and low-level programming. Follow along for insights, challenges, and deep dives into coding!

No responses yet



Write a response

What are your thoughts?

More from Abdilah CH

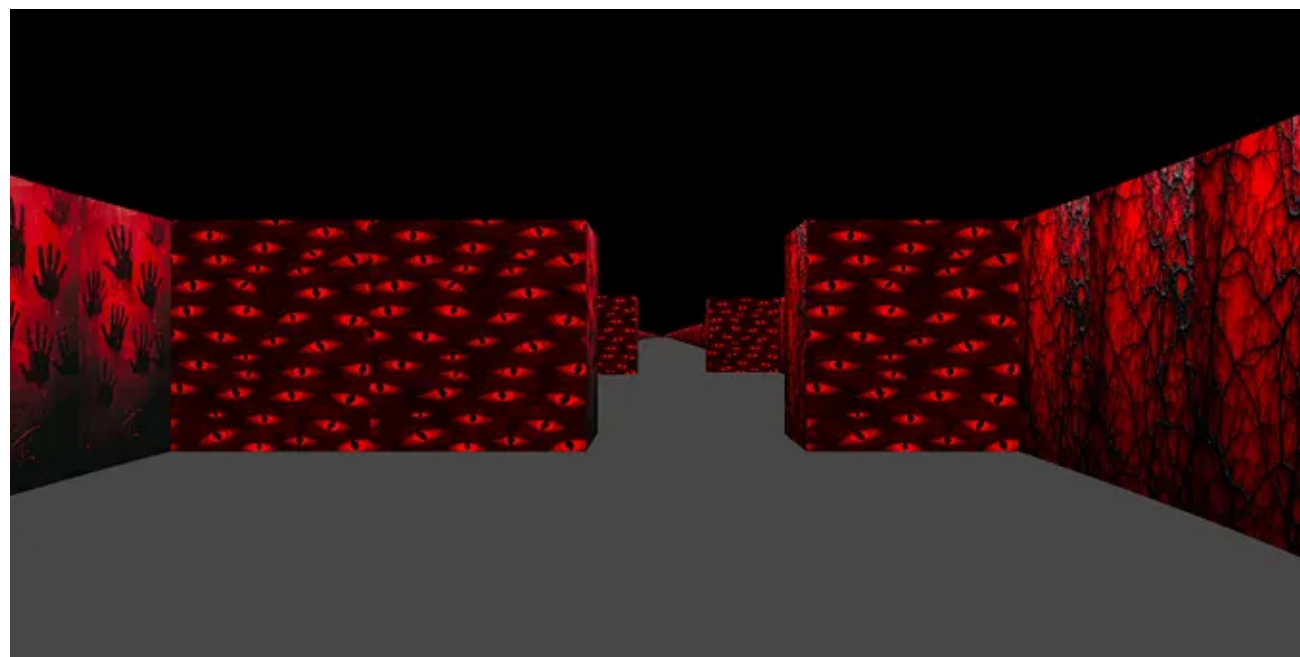


 Abdilah CH · Sep 20, 2025

File Handling in C++

File handling in C++ allows you to read from and write to files, enabling data persistence and manipulation. C++ provides the library for...

 1



 Abdilah CH · Jul 26, 2025

3D Ray-casting Game with Cub3D

 24



 Abdilah CH · Dec 18, 2025

Exception Handling in C++

1. What Is an Exception?



 Abdilah CH · Apr 25

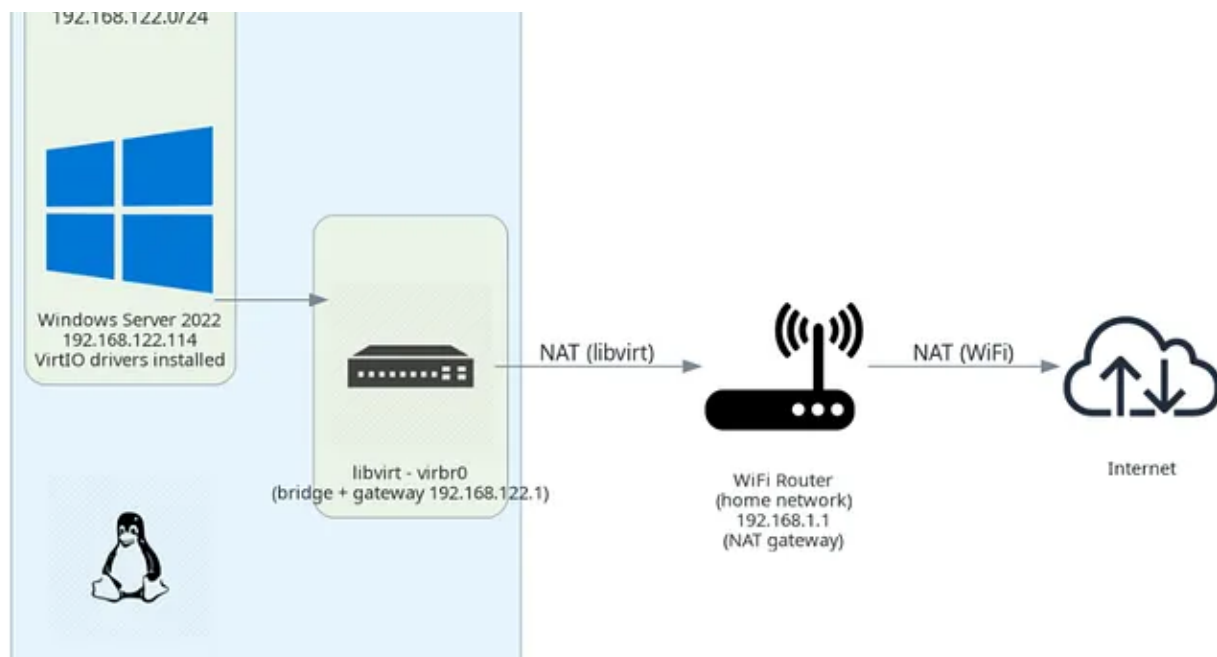
Process Lifecycle: How Programs Become Running Processes

 1



See all from Abdilah CH

Recommended from Medium



 In az azure show by Pasupathy Guru · Apr 12

HandsOn — Building Hybrid Cloud Environment — Part 1 — Identity & Connectivity Foundation

Friend link —...

★  2





Pranav Bakare · Feb 10

Understanding Linux File Permissions: Owner, Group, and Others

Linux file permissions are designed for a multi-user system where access must be controlled securely. Every file and directory belongs to...



 Abdilah CH · May 13

Inception 42: A Comprehensive Guide to Dockerizing Your First Infrastructure(PART III)

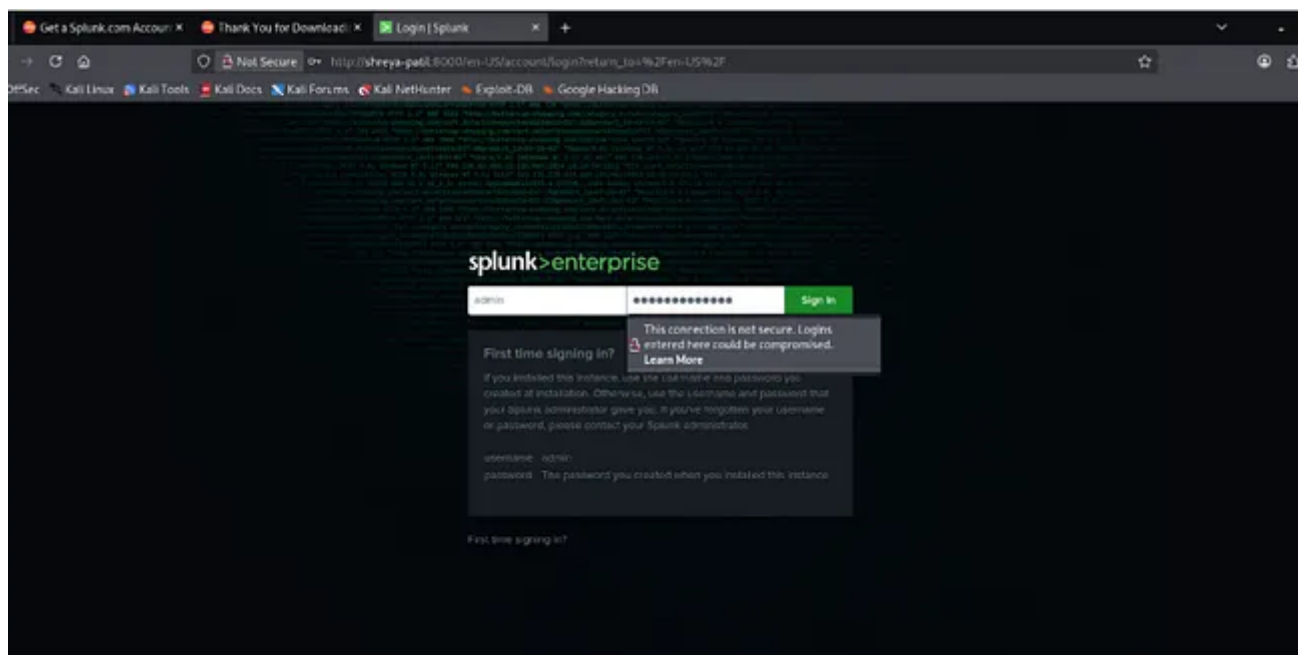


 Adel Nour · Apr 15

How You Can Make PySpark Work Across Docker, Singularity, and HPC

Why this article exists

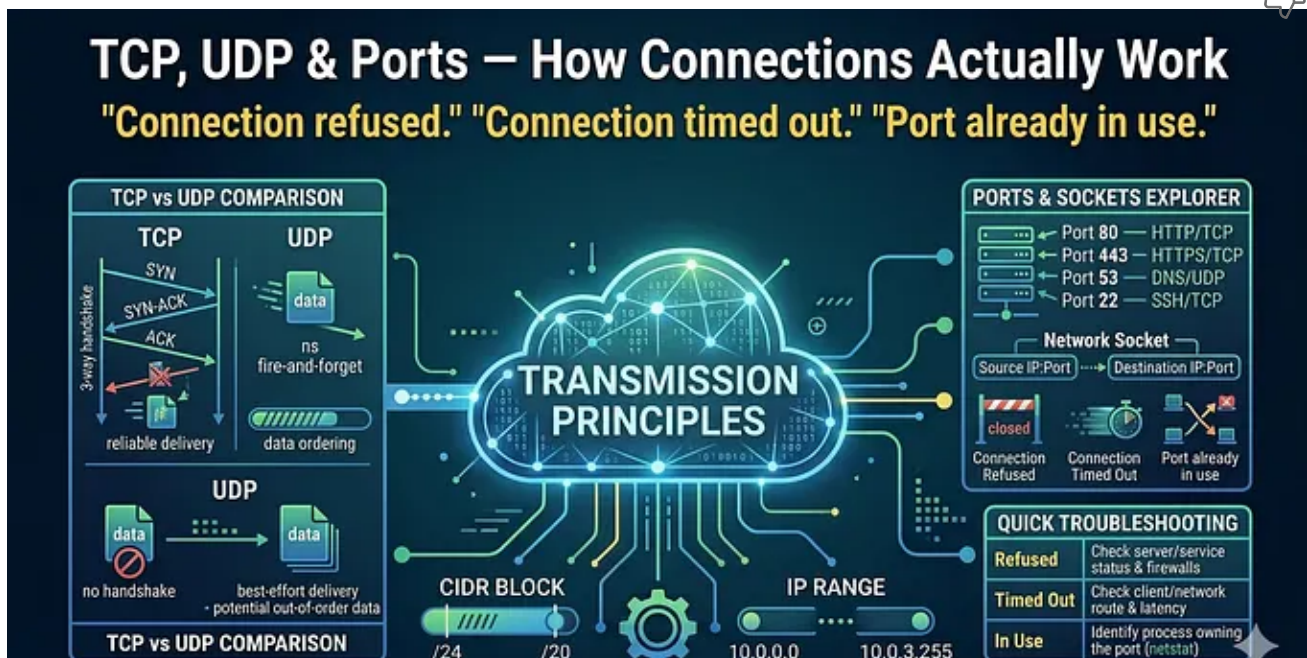
 50



S Shreya Patil · Apr 4

Understanding Process Spawing in Linux

This lab was designed to demonstrate how process execution on a Linux system can be monitored, collected, and investigated in a practical...



Mathumathiv · Mar 14

TCP, UDP & Ports — How Connections Actually Work

"Connection refused." "Connection timed out." "Port already in use."

See more recommendations